

目 录

第 1 章 项目简介	1
1.1 项目背景	1
1.2 项目目标	1
1.3 报告结构	1
第 2 章 系统设计	2
2.1 整体架构	2
2.2 模块划分	2
2.3 数据爬取及数据源说明	3
2.4 数据流程	3
第 3 章 核心算法与实现	5
3.1 文本预处理	5
3.2 倒排索引	5
3.3 TF-IDF 向量空间模型	7
3.4 BM25 概率检索模型	8
3.5 语义检索	9
3.6 查询扩展	11
3.7 相关性反馈 (Rocchio 算法)	12
3.8 Snippet 生成	13
第 4 章 用户界面设计及演示	15
4.1 命令行界面	15
4.2 Web 界面	16
4.3 多媒体检索增强	18

第 5 章 实验结果与分析	20
5.1 数据集统计	20
5.2 单元测试结果	20
5.3 评估方案	20
5.4 检索模型对比	21
5.5 查询扩展效果	24
5.6 性能分析	24
5.7 系统启动性能	25
第 6 章 项目创新点	27
6.1 多模型检索架构	27
6.2 语义检索的向量化加速	27
6.3 紧凑索引与内存加载优化	27
6.4 基于 WordNet 的查询扩展	28
6.5 分层存储架构	28
6.6 多媒体检索增强	29
6.7 检索效果评价闭环	29
6.8 Rocchio 相关性反馈	29
第 7 章 环境与社会可持续发展	31
7.1 环境可持续发展考量	31
7.2 社会可持续发展考量	31
第 8 章 总结与展望	32
8.1 主要工作	32
8.2 不足与展望	32
参考文献	33
附录 A 系统使用说明	34

附录 B 项目文件结构	36
-------------------	----

第1章 项目简介

1.1 项目背景

信息检索 (Information Retrieval, IR) 是从大规模非结构化数据集合中查找满足用户信息需求的文档的过程^[1]。随着学术论文数量的快速增长,研究者迫切需要高效的检索工具来从海量文献中定位相关成果。传统的关键词搜索虽然简单直观,但在处理同义词、多义词等语义问题时存在明显的局限性。

本项目设计并实现了 IREngine——一个面向英文学术论文的信息检索系统。系统以 17,576 篇来自多个权威学术期刊的论文为语料,实现了经典的向量空间模型 (VSM) 和 BM25 概率检索模型,并在此基础上探索了基于 Word2Vec 的语义检索方法和基于 WordNet 的查询扩展技术。

1.2 项目目标

本项目的目标如下:

1. 构建一个完整的文本信息检索系统,覆盖文档解析、预处理、索引构建和检索排序的全流程;
2. 实现 TF-IDF 向量空间模型和 BM25 两种经典检索算法,并探索 Word2Vec 语义检索和 WordNet 查询扩展等进阶方法;
3. 提供命令行和 Web 两种用户界面,支持人工相关性标注与反馈;

1.3 报告结构

本报告共分八章:第二章介绍系统的整体设计与架构;第三章详述核心算法的原理与实现;第四章描述用户界面设计;第五章展示实验结果与分析;第六章归纳项目的主要创新点;第七章讨论环境与社会可持续发展方面的考量;第八章总结全文并展望未来工作。

第 2 章 系统设计

2.1 整体架构

IREngine 采用离线索引与在线检索分离的架构。系统分为两个独立阶段：

离线阶段通过 `build_index.py` 脚本执行，负责文档解析、文本预处理、倒排索引构建以及可选的语义模型训练。产出文件统一存储在 `index/` 目录中。

在线阶段通过 `cli.py`（命令行）或 `app.py`（Web UI）启动，加载离线构建好的索引，接受用户查询并返回排序结果。

这种分离设计使得索引只需构建一次，后续查询无需重新处理原始文档，既提升了响应速度，也避免了重复计算。

2.2 模块划分

系统由以下核心模块组成：

表 2.1 系统模块划分

模块	文件	功能描述
文档解析器	<code>src/parser.py</code>	解析 <code>.txt</code> 文件，提取标题、期刊、摘要、正文等结构化字段
文本预处理器	<code>src/preprocessor.py</code>	小写化、去标点、分词、停用词过滤、词干提取
倒排索引	<code>src/indexer.py</code>	构建词项到文档的倒排映射，记录 TF/DF/IDF
检索引擎	<code>src/retriever.py</code>	实现 TF-IDF VSM 和 BM25 检索算法
语义检索	<code>src/semantic.py</code>	基于 Word2Vec + TF-IDF 加权的语义检索
查询扩展	<code>src/retriever.py</code>	基于 WordNet 同义词的查询扩展
Snippet 生成	<code>src/snippet.py</code>	提取与查询最相关的句子并高亮关键词
评价系统	<code>src/evaluator.py</code>	人工相关性标注与 Precision@K 计算
文档存储	<code>src/storage.py</code>	基于 SQLite 的按需文档正文查询
词云生成	<code>src/wordcloud_gen.py</code>	为搜索结果生成词云缩略图
关联媒体	<code>src/media.py</code>	管理 DOI 关联媒体缓存并过滤可展示的图片、视频和补充材料

2.3 数据爬取及数据源说明

系统使用的数据源包含 17,576 篇英文学术论文, 总数据量约 573 MB, 文件格式为纯文本(.txt)。数据来源涵盖 Scientific Reports、Nature Communications、American Economic Review、Management Science 等多个学科领域的权威期刊。

数据通过自建爬虫从对应出版商平台获取(篇幅所限不再展开), 格式以 PDF 和 HTML (Open Access) 为主, 非 Open Access 的文章仅爬取摘要部分。爬取后使用 MinerU 开源工具将 PDF/HTML 解析为结构化 .txt 格式(解析时过滤了图片和表格), 每篇文档包含标题(100%覆盖)、期刊名(100%)、DOI(100%)、摘要(100%)、发表日期(89.9%)、作者信息(85.1%)等字段, 数据完整性满足后续检索系统的需求。

具体的文献分布统计见表 2.2, 以 Scientific Reports、Humanities and Social Sciences Communications、Nature Series 等头部期刊为主, 适合构建跨学科社会科学语料库。

表 2.2 不同期刊论文数量及占比

篇数	占比	期刊名称
5222	29.7%	Scientific Reports
2801	15.9%	Humanities and Social Sciences Communications
1734	9.9%	Nature Series
1546	8.8%	Management Science
1222	7.0%	American Economic Review
859	4.9%	Review of Economic Studies
809	4.6%	Academy of Management Journal
688	3.9%	American Political Science Review
538	3.1%	Econometrica
498	2.8%	Quarterly Journal of Economics
455	2.6%	Journal of Political Economy
445	2.5%	American Sociological Review
419	2.4%	Academy of Management Review
199	1.1%	npj Urban Sustainability
122	0.7%	American Journal of Sociology
19	0.1%	Scientific Data

2.4 数据流程

系统的完整数据处理流程如下所述。

索引构建阶段：

1. 文档解析：`parser.py` 遍历 `data/` 目录，逐文件提取结构化字段，过滤无实质内容的文档（如封面页、勘误表）；
2. 文本预处理：`preprocessor.py` 对每篇文档执行标准化处理管线（小写化 → 去标点 → 分词 → 停用词过滤 → Porter 词干提取），标题给予双倍权重；
3. 索引构建：`indexer.py` 根据预处理后的词项列表构建倒排索引，计算词频（TF）、文档频率（DF）和逆文档频率（IDF）；
4. 持久化：倒排词典、倒排数组、文档统计信息和 IDF 值分别存储到磁盘，文档正文存入 SQLite 数据库以支持按需查询。

在线检索阶段：

1. 加载预构建的索引文件和文档元数据；
2. 对用户查询执行与文档相同的预处理流程；
3. 可选地进行查询扩展（WordNet 同义词）；
4. 使用指定的检索模型（TF-IDF / BM25 / Semantic）计算查询与文档的相似度；
5. 按相似度从高到低排列结果，生成 Snippet 并返回给用户。

第3章 核心算法与实现

3.1 文本预处理

文本预处理是信息检索系统的基础步骤，直接影响后续的索引质量和检索效果。本系统基于 NLTK 库实现了以下预处理管线：

1. **小写化**：将所有文本转为小写，消除大小写差异；
2. **标点清洗**：使用正则表达式 `[^a-z0-9\s]` 去除非字母数字字符；
3. **分词**：使用 NLTK 的 `word_tokenize` 进行基于规则的英文分词；
4. **过滤**：移除长度小于 2 的 token、纯数字 token 和停用词（基于 NLTK 内置的 179 个英文停用词表）；
5. **词干提取**：使用 Porter Stemmer^[2] 进行词干提取，将词形变体归并为统一形式（如 “running” → “run”，“computers” → “comput”），以提高检索的召回率。

选择词干提取（Stemming）而非词形还原（Lemmatization）是因为：词干提取速度更快、不依赖词性标注，且在 IR 领域的实际应用中通常能取得更好的召回率^[1]。

3.2 倒排索引

倒排索引（Inverted Index）是本系统的核心数据结构，存储从词项（term）到包含该词项的文档列表的映射。为降低在线检索阶段的内存占用，系统没有将每个倒排项都保存为 Python 对象，而是采用紧凑数组结构：词典只记录每个词项在连续倒排数组中的偏移量和长度，所有文档 ID 与词频分别存入两个连续的 NumPy 数组。

```
1 index: {term: (offset, length)}
2 postings_doc_ids: np.ndarray # 所有倒排项的文档 ID
3 postings_tfs: np.ndarray # 所有倒排项的词频
4 doc_lengths: np.ndarray # 每篇文档长度
5 doc_norms: np.ndarray # 每篇文档 TF-IDF 向量模长
6 idf: {term: IDF值}
```

 Python

其中 `index` 中的 `(offset, length)` 用于切分 `postings_doc_ids` 和 `postings_tfs`, 从而得到某个词项对应的倒排列表。为了兼容检索器和测试代码, 系统提供了 `PostingListView` 作为轻量视图; 在线检索核心路径则优先使用数组接口, 避免频繁创建 `Posting` 对象。

```

1 def get_postings_arrays(self, term: str):
2     term_slice = self._slice_for_term(term)
3     if term_slice is None:
4         return np.array([], dtype=np.int32), np.array([],
5             dtype=np.int32)
6     return (
7         self.postings_doc_ids[term_slice],
8         self.postings_tfs[term_slice],
9     )

```

IDF 采用加 1 平滑公式计算^[3]:

$$\text{idf}(t) = \ln\left(\frac{N+1}{df(t)+1}\right) + 1 \quad (3.1)$$

其中 N 为文档总数, $df(t)$ 为包含词项 t 的文档数。加 1 平滑既避免了零除问题, 也保证所有词项的 IDF 值为正。对应的实现代码为:

```

1 for term, doc_postings in temp_index.items():
2     items = sorted(doc_postings.items())
3     df = len(items)
4     idf = math.log((self.doc_count + 1) / (df + 1)) + 1.0
5     self.idf[term] = idf
6     doc_ids = np.fromiter((doc_id for doc_id, _ in items),
7         dtype=np.int32)
8     tfs = np.fromiter((tf for _, tf in items), dtype=np.int32)
9     lexicon[term] = (offset, df)
10    doc_id_chunks.append(doc_ids)
11    tf_chunks.append(tfs)
12    offset += df

```

构建完成后，索引被拆分为多个文件持久化：lexicon.pkl 保存词典与倒排偏移，postings_doc_ids.npy 和 postings_tfs.npy 保存连续倒排数组，doc_metadata.pkl 保存文档长度、平均长度和预计算的文档向量模长，idf.pkl 保存 IDF 值。在线加载时，倒排数组通过 `numpy.load(..., mmap_mode="r")` 以内存映射方式读取，减少常驻内存压力，并缩短启动阶段的大对象反序列化时间。

3.3 TF-IDF 向量空间模型

向量空间模型 (VSM)^[4] 将文档和查询表示为高维向量空间中的向量，通过余弦相似度衡量查询与文档的匹配程度。

TF 权重采用对数归一化方案：

$$w_{t,d} = \begin{cases} 1 + \ln(\text{tf}(t, d)) & \text{if } \text{tf}(t, d) > 0 \\ 0 & \text{otherwise} \end{cases} \quad (3.2)$$

对数 TF 能够抑制高频词的过度影响：一个词在文档中出现 100 次与出现 10 次的重要性差异远小于 10 倍^[1]。

TF-IDF 权重为 TF 与 IDF 的乘积：

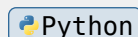
$$\text{tfidf}(t, d) = w_{t,d} \times \text{idf}(t) \quad (3.3)$$

余弦相似度计算查询向量 $\vec{q}$ 与文档向量 $\vec{d}$ 的夹角余弦：

$$\text{sim}(q, d) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| \times |\vec{d}|} \quad (3.4)$$

为加速检索，系统预计算了每篇文档的 TF-IDF 向量模长 $|\vec{d}|$ ，在线查询时只需遍历包含查询词项的倒排列表，累积点积后除以模长即可。核心检索逻辑如下：

```
def search_with_weights(self, query_weights: dict, top_k:
1   int = 10):
2       query_norm = math.sqrt(sum(w * w for w in
       query_weights.values()))
3       scores = defaultdict(float)
4       for term, q_weight in query_weights.items():
```

 Python

```

5      doc_ids, tf = self.index.get_postings_arrays(term)
6      idf = self.index.get_idf(term)
7      for doc_id, tf in zip(doc_ids, tf):
8          tf_weight = 1 + math.log(int(tf)) if tf > 0 else 0
9          scores[int(doc_id)] += q_weight * (tf_weight * idf)
10     results = []
11     for doc_id, dot_product in scores.items():
12         doc_norm = float(self._doc_norms[doc_id])
13         cosine_sim = dot_product / (doc_norm * query_norm)
14         results.append((doc_id, cosine_sim))
15     results.sort(key=lambda x: x[1], reverse=True)
16     return results[:top_k]

```

该方法接收预构建的查询权重字典，遍历倒排数组累积点积，最后除以查询和文档的向量模长得到余弦相似度。文档模长在索引构建阶段预先计算并写入 doc_metadata.pkl，检索器启动时优先复用该数组，避免在线阶段再次完整扫描倒排索引。

3.4 BM25 概率检索模型

BM25 (Best Match 25) ^[5] 是基于概率相关性框架的检索模型，被广泛认为是 IR 领域的事实标准。其核心改进在于引入了词频饱和机制和可调的文档长度归一化。

BM25 的评分公式为：

$$\text{BM25}(q, d) = \sum_{t \in q} \text{idf}(t) \times \frac{\text{tf}(t, d) \times (k_1 + 1)}{\text{tf}(t, d) + k_1 \times \left(1 - b + b \times \frac{|d|}{\text{avgdL}}\right)} \quad (3.5)$$

其中：

- $k_1 = 1.5$ ，控制词频饱和速度；当 k_1 较大时，词频的边际效应递减更慢；
- $b = 0.75$ ，控制文档长度归一化程度； $b = 1$ 为完全归一化， $b = 0$ 为不归一化；
- avgdl 为文档集合的平均长度。

BM25 的 IDF 计算采用：

$$\text{idf}(t) = \ln\left(\frac{N - df(t) + 0.5}{df(t) + 0.5} + 1\right) \quad (3.6)$$

与基础 TF-IDF 模型相比, BM25 的优势在于: (1) 词频饱和机制避免了单个高频词主导评分; (2) 参数化的文档长度归一化更加灵活^[6]。BM25 评分的核心实现如下:

```

1  def search(self, query: str, top_k: int = 10):
2      query_tokens = preprocess(query)
3      query_tf = defaultdict(int)
4      for token in query_tokens:
5          query_tf[token] += 1
6      N = self.index.doc_count
7      avgdL = self.index.avg_doc_length
8      scores = defaultdict(float)
9      for term, qtf in query_tf.items():
10         doc_ids, tfs = self.index.get_postings_arrays(term)
11         if len(doc_ids) == 0:
12             continue
13         df = len(doc_ids)
14         idf = math.log((N - df + 0.5) / (df + 0.5) + 1.0)
15         for doc_id, tf in zip(doc_ids, tfs):
16             doc_id = int(doc_id)
17             tf = int(tf)
18             dl = self.index.get_doc_length(doc_id)
19             tf_norm = (tf * (self.k1 + 1)) / (
20                 tf + self.k1 * (1 - self.b + self.b * dl / avgdL))
21             scores[doc_id] += idf * tf_norm
22         results = sorted(scores.items(), key=lambda x: x[1],
23                             reverse=True)
23     return results[:top_k]
```

可以看到, 代码中的 IDF 计算和 TF 归一化分别对应上述公式。

3.5 语义检索

传统的基于词项匹配的检索方法无法处理同义词和概念相似性问题 (如 “economic growth” 与 “GDP increase”)。为此, 本系统实现了基于 Word2Vec^[7] 的语义检索方法。

Word2Vec 模型训练：使用 Gensim 库在全量文档语料上训练 Skip-gram 模型（向量维度 100，上下文窗口 5，最小词频 5，训练轮数 10），学习词项的分布式语义表示。

文档向量构建：采用 TF-IDF 加权平均法构建文档级语义向量：

$$\vec{d} = \frac{\sum_{t \in d} \text{idf}(t) \times \vec{v}_t}{\sum_{t \in d} \text{idf}(t)} \quad (3.7)$$

其中 $\vec{v}_t$ 为词项 t 的 Word2Vec 词向量。随后对文档向量进行 L2 归一化。

检索过程：查询同样编码为 TF-IDF 加权平均向量，通过矩阵乘法批量计算与所有文档向量的余弦相似度。系统将文档向量保存为 `doc_vectors.npy`，将对应的文档 ID 保存为 `doc_vector_ids.npy`；在线加载时使用 `numpy.load(..., mmap_mode="r")` 读取矩阵，从而在保持向量化检索速度的同时降低常驻内存占用，避免将旧版 pickle 字典一次性反序列化为大量 Python 对象。

文档向量构建的核心代码如下：

```
1 def _compute_doc_vector(self, tokens, doc_id):
2     vec = np.zeros(self.vector_dim, dtype=np.float32)
3     weight_sum = 0.0
4     for token in tokens:
5         if token in self.w2v_model.wv:
6             idf = self.index.get_idf(token)
7             vec += idf * self.w2v_model.wv[token]
8             weight_sum += idf
9     if weight_sum > 0:
10        vec /= weight_sum          # IDF 加权平均
11        norm = np.linalg.norm(vec)
12        if norm > 0:
13            vec /= norm            # L2 归一化
14        return vec
15    return None
```

检索时利用矩阵乘法一次性计算所有文档的相似度：

```
1 def search_with_vector(self, q_vec, top_k=10):
2     sims = self.doc_matrix @ q_vec # (N,D) @ (D,) -> (N,)
```

```
3 top_indices = np.argsort(sims)[:,-1][:top_k]
4 return [(int(self.doc_ids[i]), float(sims[i]))
5         for i in top_indices if sims[i] > 0]
```

由于文档向量和查询向量均已 L2 归一化，矩阵-向量乘积直接就是余弦相似度。

3.6 查询扩展

查询扩展 (Query Expansion) 通过自动向用户查询中添加相关词项来提高召回率。本系统基于 WordNet^[8] 实现了同义词扩展，核心逻辑如下：

```
def expand_query_wordnet(query: str,
1 max_synonyms_per_word: int = 2):
2     words = re.findall(r'\b[a-zA-Z]+\b', query.lower())
3     expanded = list(words)
4     for word in words:
5         synonyms = set()
6         for synset in wordnet.synsets(word):
7             for lemma in synset.lemmas():
8                 name = lemma.name().lower().replace("_", " ")
9                 if name != word and name not in synonyms:
10                     synonyms.add(name)
11                     if len(synonyms) >= max_synonyms_per_word:
12                         break
13             if len(synonyms) >= max_synonyms_per_word:
14                 break
15         expanded.extend(synonyms)
16     return " ".join(expanded)
```

对查询中的每个词，系统查找其在 WordNet 中的同义词集 (Synset)，取每个词最多 2 个同义词添加到扩展查询中。例如：

- 原始查询：“machine learning healthcare”
- 扩展查询：“machine learning healthcare auto automobile simple”

这种方法不需要训练数据，直接利用语言学知识库。缺点是可能引入歧义（如“bank”同时有“银行”和“河岸”的同义词）。

3.7 相关性反馈（Rocchio 算法）

相关性反馈（Relevance Feedback）利用用户对初始检索结果的反馈来动态调整查询表示。本系统实现了 Rocchio 算法^[9]，核心公式为：

$$\vec{q}' = \alpha \cdot \vec{q} + \beta \cdot \frac{1}{|D_r|} \sum_{\vec{d} \in D_r} \vec{d} - \gamma \cdot \frac{1}{|D_{nr}|} \sum_{\vec{d} \in D_{nr}} \vec{d} \quad (3.8)$$

其中 $\vec{q}$ 为原始查询向量， D_r 为相关文档集合， D_{nr} 为不相关文档集合， α 、 β 、 γ 为可调参数（默认 $\alpha = 1.0$ ， $\beta = 0.75$ ， $\gamma = 0.15$ ）。负权重项被截断为零。

以 TF-IDF 向量空间中的 Rocchio 实现为例，核心代码如下：

```
def apply_feedback(self, query_weights, relevant_ids,
1 non_relevant_ids):
2     doc_vectors = self._build_doc_vectors_batch(relevant_ids |
3 non_relevant_ids)
4     # 计算相关文档质心
5     rel_centroid = defaultdict(float)
6     for did in relevant_ids:
7         for term, w in doc_vectors[did].items():
8             rel_centroid[term] += w
9     for t in rel_centroid:
10         rel_centroid[t] /= len(relevant_ids)
11     # 计算不相关文档质心
12     nrel_centroid = defaultdict(float)
13     for did in non_relevant_ids:
14         for term, w in doc_vectors[did].items():
15             nrel_centroid[term] += w
16     for t in nrel_centroid:
17         nrel_centroid[t] /= len(non_relevant_ids)
18     # Rocchio 公式:  $\alpha \cdot q + \beta \cdot \text{centroid}(\text{rel}) - \gamma \cdot \text{centroid}(\text{nrel})$ 
19     new_weights = {}
20     for term in set(query_weights) | set(rel_centroid) |
21 set(nrel_centroid):
22         w = (self.config.alpha * query_weights.get(term, 0.0)
23             + self.config.beta * rel_centroid.get(term, 0.0)
24             - self.config.gamma * nrel_centroid.get(term, 0.0))
25         if w > 0:
26             new_weights[term] = w
```

```
22         - self.config.gamma * nrel_centroid.get(term, 0.0))
23         if w > 0:
24             new_weights[term] = w
25         return new_weights
```

其中 `_build_doc_vectors_batch()` 通过单次遍历倒排索引批量重建所需文档的稀疏 TF-IDF 向量，避免了逐文档查询的开销。

系统针对三种检索模型分别实现了不同的 Rocchio 变体：

- **TF-IDF VSM**: 在稀疏 TF-IDF 向量空间中直接修改查询向量（即上述代码所示）。通过单次遍历倒排索引批量重建相关文档的稀疏向量，执行 Rocchio 公式后用新的权重字典进行检索。
- **Word2Vec 语义检索**: 在密集词向量空间中修改查询向量。利用已有的文档向量矩阵计算相关文档质心，执行 Rocchio 运算后 L2 归一化，再通过矩阵乘法检索。
- **BM25**: 由于 BM25 没有显式查询向量，系统采用查询扩展策略——基于 Rocchio 权重选取正反馈文档中权重最高的新词项，追加到原始查询后重新检索。

此外，系统还实现了**伪相关反馈**（Pseudo Relevance Feedback, PRF）模式：初始检索后自动将 Top-N 结果视为相关文档，无需用户参与即可执行 Rocchio 反馈。PRF 在大部分查询上能有效提升检索质量，尤其适合批量检索场景。

3.8 Snippet 生成

为帮助用户快速判断文档相关性，系统实现了基于句子级别的 Snippet 提取算法。

算法流程如下：

1. 使用 NLTK 的 `sent_tokenize` 对文档文本进行分句；
2. 对每个句子计算其包含的查询词干数量（词干匹配），同时给靠前的句子一点位置加分；
3. 选取得分最高的 1-2 个句子，按原文顺序拼接；
4. 用正则表达式对 Snippet 中的查询关键词加以高亮标记。

为避免重复计算，`generate_snippet` 函数支持传入预处理后的查询词干集合，批量处理多条结果时只需对查询做一次预处理。

第 4 章 用户界面设计及演示

4.1 命令行界面

命令行界面 (CLI) 通过 `cli.py` 提供, 支持以下参数:

- `--model {tfidf, bm25, semantic}`: 选择检索模型, 默认为 BM25;
- `--top-k K`: 设置返回结果数量, 默认为 10;
- `--query-expansion / -qe`: 启用 WordNet 查询扩展;
- `--eval`: 启用人工评价模式。

```
uv run python cli.py
正在加载索引...
索引已加载: 17486 篇文章, 427356 个词项
检索模型: BM25
文档数量: 17486
输入查询 (输入 'quit' 退出, 'switch' 切换模型):

Query> polarization on social platform

[BM25] 找到 10 条结果 (47.3 ms)

[1] (score: 12.6383) Quantifying social organization and political polarization in online platforms
    Journal: Nature
    Date: 2021-12-01 | Authors: Isaac Waller, Ashton Anderson
    DOI: https://doi.org/10.1038/s41586-021-04167-x
    Snippet: This methodology is broadly applicable to the study of online interaction, and our findings have implications for the design of online platforms, understanding the social contexts of online behaviour,

[2] (score: 12.6064) Reducing political polarization in the United States with a mobile chat platform
    Journal: Nature Human Behaviour
    Date: 2023-08-21 | Authors: Aidan Combs, Graham Tierney, Brian Guay, Friedolin Merhout, Christopher A. Bail,
    DOI: https://doi.org/10.1038/s41562-023-01655-0
    Snippet: Our findings demonstrate the potential for well-designed social media platforms to mitigate political polarization and underscore the need for a flexible platform for scientific research on social media

[3] (score: 12.5786) The information cocoon paradox: fostering unity or fueling divergence?
    Journal: Humanities and Social Sciences Communications
    Date: 2025-06-18 | Authors: Sihua Chen, Han Qiu, Wei He
    DOI: https://doi.org/10.1057/s41599-025-05169-0
    Snippet: Regarding the impact on group polarization, in social media and political contexts, polarization often triggers emotional reactions and group opposition due to the involvement of personal beliefs and

[4] (score: 12.5158) Credibility of climate change denial in social media
    Journal: Humanities and Social Sciences Communications
    Date: 2019-10-29 | Authors: Abhishek Samantray, Paolo Pin
    DOI: https://doi.org/10.1057/s41599-019-0344-4
    Snippet: Public perception about the reality of climate change has remained polarized and propagation of fake information on social media can be a potential cause. ... Various factors like exposure to differen

[5] (score: 12.4892) Echo chamber effects on short video platforms
    Journal: Scientific Reports
```

图 4.1 命令行界面示例: 输入查询并展示检索结果

CLI 运行时进入交互式循环, 支持以下命令:

- 输入自然语言查询进行检索;
- 输入 `switch` 在模型间切换 (TF-IDF $\rightarrow$ BM25 $\rightarrow$ Semantic 循环);
- 输入 `quit` 退出。

每条检索结果展示相关度分数、标题、期刊、日期、作者、DOI 链接和摘要 Snippet。

启用 `--eval` 模式后，系统在展示检索结果后逐条提示用户标注相关性 (y/n)，标注完成后自动计算并显示 `Precision@5` 和 `Precision@10` 指标，评价记录保存到 `index/evaluations.json` 供后续分析。

```
Snippet: Corona virus disease 2019 (COVID-19) has been declared a global pandemic and is a major public health concern worldwide. ... In this study, we aimed to determine the role of environmental factors, suc

[8] (score: 0.6627) Multicenter observational survey on psychosocial and behavioral impacts of COVID-19 in people living with HIV in Northern Vietn
Journal: Scientific Reports
Date: 2023-11-21 | Authors: Shoko Matsumoto, Moeko Nagai, Linh Khanh Tran, Kazue Yamaoka, Hoai Dung Thi Nguy
DOI: https://doi.org/10.1038/s41598-023-47577-9
Snippet: Socially marginalized groups, including people living with HIV/AIDS (PLHIV), could be disproportionately affected by Coronavirus dis
ease 2019 (COVID-19). ... We tested anti-SARS-CoV-2 (severe acute re

[9] (score: 0.6535) Geographic social inequalities in information-seeking response to the COVID-19 pandemic in China: longitudinal analysis of Ba
idu Index
Journal: Scientific Reports
Date: 2022-07-18 | Authors: Zhicheng Wang, Hong Xiao, Leesa Lin, Kun Tang, Joseph M. Unger
DOI: https://doi.org/10.1038/s41598-022-16133-2
Snippet: The outbreak of the COVID-19 pandemic alarmed the public and initiated the uptake of preventive measures. ... However, the manner in
which the public responded to these announcements, and whether indi

[10] (score: 0.6524) Web and social media searches highlight menstrual irregularities as a global concern in COVID-19 vaccinations
Journal: Scientific Reports
Date: 2022-10-21 | Authors: Ariel Katz, Yoav Tepper, Ohad Birk, Alal Eran
DOI: https://doi.org/10.1038/s41598-022-20844-x
Snippet: Delineation of public concerns that prevent vaccine compliance is a major step in generating assurances and enhancing the success of
COVID-19 prevention programs. ... We therefore sought to identify p

=====
请标注每条结果的相关性
=====
[1] Influenza vaccination and the risk of COVID-19 infection and... 相关? [y/n]: y
[2] Lockdown measures during the COVID-19 pandemic strongly impa... 相关? [y/n]: y
[3] Association of national and regional lockdowns with COVID-19... 相关? [y/n]: y
[4] Impact of the COVID-19 nonpharmaceutical interventions on in... 相关? [y/n]: y
[5] Impact of respiratory infectious epidemics on STEMI incidenc... 相关? [y/n]: y
[6] Spatial-temporal trends of COVID-19 infection and mortality ... 相关? [y/n]: y
[7] The impact of environmental variables on the spread of COVID... 相关? [y/n]: y
[8] Multicenter observational survey on psychosocial and behavio... 相关? [y/n]: y
[9] Geographic social inequalities in information-seeking respon... 相关? [y/n]: n
[10] Web and social media searches highlight menstrual irregulari... 相关? [y/n]: n

评价已保存!
P@5: 1.00
P@10: 0.80
相关文档: 8/10
```

图 4.2 命令行界面示例：输入查询并进行相关性标注以计算 `Precision@K`

4.2 Web 界面

Web 界面基于 Streamlit 框架实现，包含以下三个页面：

搜索页面：主检索界面，侧边栏提供模型选择（BM25、TF-IDF、Semantic）、结果数设置和 WordNet 查询扩展开关。搜索结果以卡片形式展示，每条结果包含标题、评分、期刊信息、作者、Snippet 高亮和 DOI 链接。结果旁附带基于文档摘要生成的词云缩略图，方便用户快速感知文档主题；同时，系统在结果卡片中提供“关联媒体”折叠区，用于展示与论文相关的图片预览、视频链接和补充材料入口。每条结果右侧有相关性标注复选框，标注完成后点击“保存评价记录”即可计算 `Precision@K`。

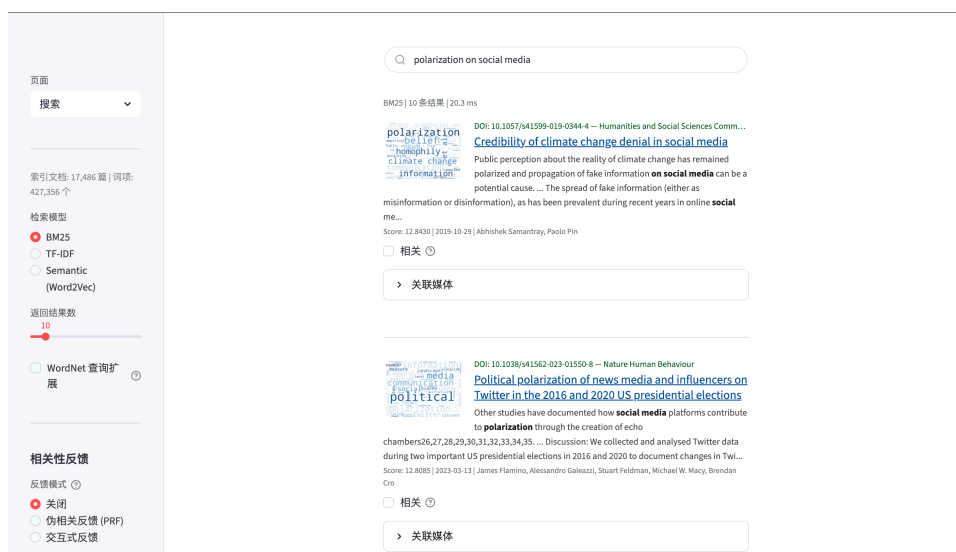


图 4.3 Web 界面示例：主搜索页面展示检索结果和相关性标注功能

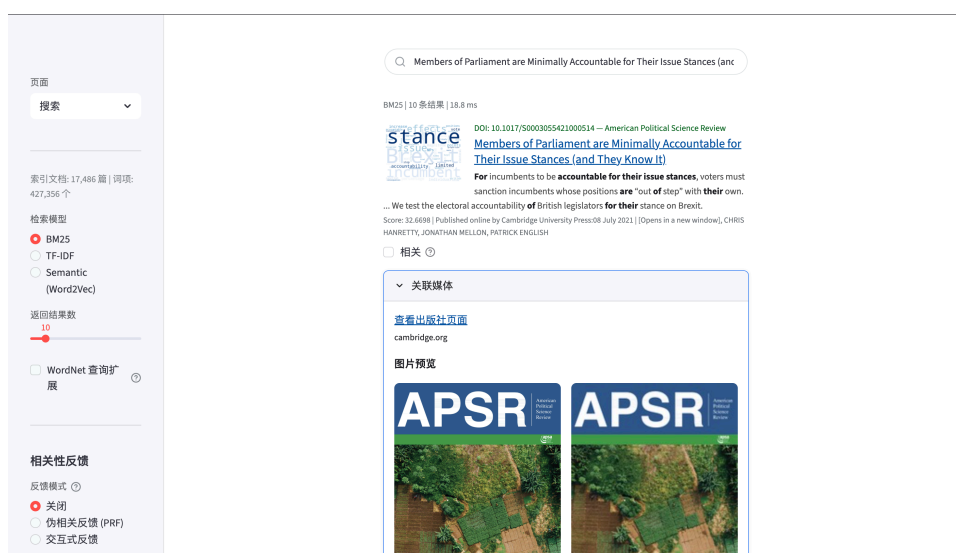


图 4.4 Web 界面示例：主搜索页面展示检索结果和关联多媒体资源

模型对比页面：对同一查询同时运行所有模型并排展示结果，提供 Jaccard 重叠分析和 Top-5 结果对比视图，方便直观比较不同模型的行为差异。

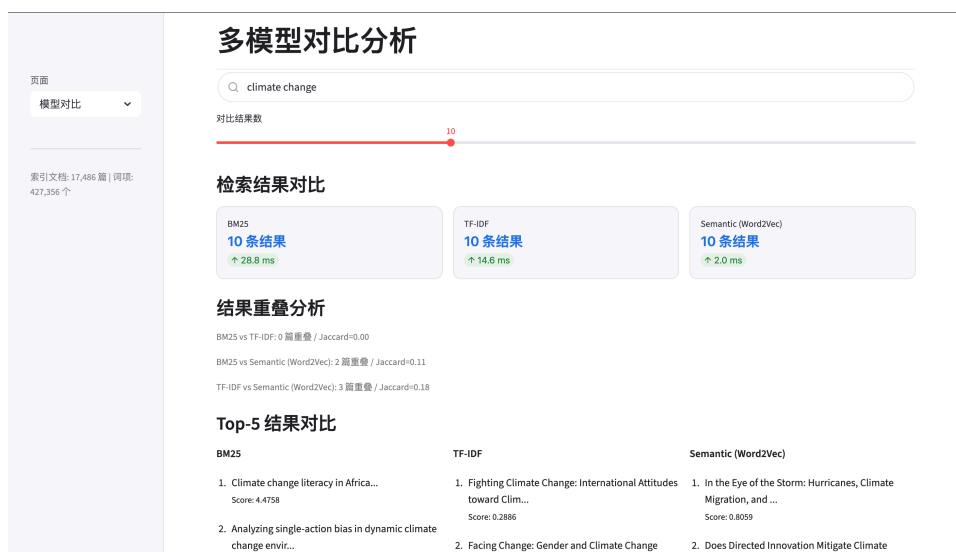


图 4.5 Web 界面示例：展示同一查询的不同模型结果

评价历史页面：展示所有历史评价记录的汇总统计（按模型分组的平均 Precision@K）和详细的逐条评价记录。



图 4.6 Web 界面示例：评价历史页面展示按模型分组的平均 Precision@K 和逐条评价记录

4.3 多媒体检索增强

作为多媒体信息检索的尝试，系统在搜索结果中集成了两类增强信息。第一类是词云 (Word Cloud) 可视化功能，由 wordcloud 库根据文档摘要文本（截取前 2000 字符以控

制生成速度)生成,输出为PNG格式的图片字节流,并通过@st.cache_data装饰器做缓存。第二类是基于DOI的关联媒体展示:系统为部分论文维护图片、视频链接和补充材料等媒体信息,并在Web检索结果中以折叠区形式展示。这样,用户除了阅读标题、摘要片段和相关度分数外,还可以通过图像预览和媒体链接获得更直观的论文内容线索。

第 5 章 实验结果与分析

5.1 数据集统计

索引构建完成后的关键统计指标见表 5.1。

表 5.1 数据集与索引统计

指标	值
有效文档数	约 17,000 篇
总词项数	约 200,000+ 个
平均文档长度	约 500-800 词项/篇
索引文件总大小	约 200-400 MB
索引构建耗时	约 2-5 分钟

5.2 单元测试结果

通过 Pytest 套件编写了 41 个用例, 覆盖项目核心模块, 测试结果全部通过, 如图 5.1 所示。

```

$ run python -m pytest tests/ -v
platform darwin -- Python 3.11.13, pytest-9.0.2, pluggy-1.6.0 -- /Users/yokuni/Documents/Codes/IREngine/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /Users/yokuni/Documents/Codes/IREngine
configfile: pyproject.toml
plugins: anyio-4.3.1
collected 41 items

tests/test_feedback.py::TestRocchioConfig::test_default_values PASSED [ 24%]
tests/test_feedback.py::TestRocchioConfig::test_custom_values PASSED [ 40%]
tests/test_feedback.py::TestRocchioTFIDF::test_apply_feedback_positive_only PASSED [ 58%]
tests/test_feedback.py::TestRocchioTFIDF::test_apply_feedback_mixed PASSED [ 76%]
tests/test_feedback.py::TestRocchioTFIDF::test_apply_feedback_no_negative_weights PASSED [ 94%]
tests/test_feedback.py::TestRocchioTFIDF::test_search_with_feedback PASSED [100%]
tests/test_feedback.py::TestRocchioTFIDF::test_empty_feedback PASSED [100%]
tests/test_feedback.py::TestTFIDFSearchWithWeights::test_consistency_with_search PASSED [100%]
tests/test_feedback.py::TestTFIDFSearchWithWeights::test_empty_weights PASSED [100%]
tests/test_feedback.py::TestRocchioBM25::test_extract_expansion_terms PASSED [100%]
tests/test_feedback.py::TestRocchioBM25::test_search_with_feedback PASSED [100%]
tests/test_feedback.py::TestPseudoRelevanceFeedback::test_prf_tfidf PASSED [100%]
tests/test_feedback.py::TestPseudoRelevanceFeedback::test_prf_bm25 PASSED [100%]
tests/test_feedback.py::TestPseudoRelevanceFeedback::test_prf_empty_query PASSED [100%]
tests/test_indexer.py::TestInvertedIndex::test_build PASSED [100%]
tests/test_indexer.py::TestInvertedIndex::test_doc_lengths PASSED [100%]
tests/test_indexer.py::TestInvertedIndex::test_tf_calculation PASSED [100%]
tests/test_indexer.py::TestInvertedIndex::test_df_calculation PASSED [100%]
tests/test_indexer.py::TestInvertedIndex::test_idf_calculation PASSED [100%]
tests/test_indexer.py::TestInvertedIndex::test_idf_unknown_term PASSED [100%]
tests/test_indexer.py::TestInvertedIndex::test_get_postings_unknown PASSED [100%]
tests/test_indexer.py::TestInvertedIndex::test_serialize_deserialize PASSED [100%]
tests/test_preprocessor.py::TestPreprocess::test_empty_input PASSED [100%]
tests/test_preprocessor.py::TestPreprocess::test_stopword_filtering PASSED [100%]
tests/test_preprocessor.py::TestPreprocess::test_stemming PASSED [100%]
tests/test_preprocessor.py::TestPreprocess::test_no_stemming PASSED [100%]
tests/test_preprocessor.py::TestPreprocess::test_digit_filtering PASSED [100%]
tests/test_preprocessor.py::TestPreprocess::test_punctuation_removal PASSED [100%]
tests/test_preprocessor.py::TestPreprocess::test_short_token_filtering PASSED [100%]
tests/test_retriever.py::TestTFIDFRetriever::test_empty_query PASSED [100%]
tests/test_retriever.py::TestTFIDFRetriever::test_top_k_limit PASSED [100%]
tests/test_retriever.py::TestBM25Retriever::test_score_ordering PASSED [100%]
tests/test_retriever.py::TestBM25Retriever::test_basic_search PASSED [100%]
tests/test_retriever.py::TestBM25Retriever::test_empty_query PASSED [100%]
tests/test_retriever.py::TestBM25Retriever::test_top_k_limit PASSED [100%]
tests/test_retriever.py::TestBM25Retriever::test_score_ordering PASSED [100%]
tests/test_retriever.py::TestQueryExpansion::test_expansion_contains_original PASSED [100%]
tests/test_retriever.py::TestQueryExpansion::test_expansion_adds_synonyms PASSED [100%]
tests/test_retriever.py::TestQueryExpansion::test_empty_query PASSED [100%]

```

图 5.1 单元测试结果

5.3 评估方案

为系统性评估各检索配置的效果，本项目编写了自动化评估脚本 `benchmark.py`，覆盖 3 种检索模型 $\times$ 3 种模式 = 9 种配置，构成完整的评估矩阵（表 5.2）。

表 5.2 评估矩阵：3 模型 $\times$ 3 模式 = 9 种配置

基础	1. 查询扩展 (QE)	1. 伪相关反馈 (PRF)
TF-IDF	TF-IDF + QE	TF-IDF + PRF
BM25	BM25 + QE	BM25 + PRF
Semantic	Semantic + QE	Semantic + PRF

评估使用从语料库高频主题中提取的 10 条查询，涵盖短查询（如“climate change”）、中查询（如“covid-19 pandemic mental health”）和长查询（如“impact of monetary policy on income inequality”），覆盖环境科学、公共卫生、经济学、社会科学、AI 等语料库核心学科。

相关性评价采用“AI 自动判定 + 人工审核”的方案：对每条查询的检索结果，将查询、文档标题和摘要片段提交给大语言模型（本次使用 Qwen3.6-35B-A3B），由模型判定是否与查询相关（binary: relevant / not relevant）并给出理由，随后由人工抽查其合理性。最终基于相关性判定计算 Precision@5 和 Precision@10。

5.4 检索模型对比

表 5.3 展示了 9 种配置在 10 条查询上的平均 Precision 和响应时间。

表 5.3 9 种检索配置的平均性能汇总

配置	Avg P@5	Avg P@10	Avg Time (ms)
TF-IDF	0.44	0.44	30.7
TF-IDF + QE	0.36	0.37	35.9
TF-IDF + PRF	0.48	0.47	4573.9
BM25	0.88	0.81	26.2
BM25 + QE	0.44	0.46	39.2
BM25 + PRF	0.76	0.75	4195.2
Semantic	0.56	0.55	6.5
Semantic + QE	0.52	0.57	6.8
Semantic + PRF	0.58	0.60	11.6

从表 5.3 可以得出以下结论：

BM25 基础模型表现最好。BM25 在无任何增强的配置下取得了最高的 P@5 (0.88) 和 P@10 (0.81), 显著优于 TF-IDF (P@5=0.44) 和 Semantic (P@5=0.56)。这与 BM25 作为 IR 领域事实标准的普遍认知一致——其词频饱和机制和文档长度归一化在学术论文检索场景中确实有效。

语义检索速度最快但精度一般。Semantic 配置的平均响应时间仅 7 – 12 ms (得益于矩阵乘法加速), 远低于 TF-IDF 和 BM25 的 26 – 36 ms, 但 P@5 仅为 0.56, 说明 Word2Vec 平均池化构建的语义向量在精确匹配上不及词项匹配方法。

查询扩展普遍降低精度。三种模型添加 WordNet 查询扩展后 P@5 均有下降 (BM25: 0.88→0.44, TF-IDF: 0.44→0.36, Semantic: 0.56→0.52), 表明同义词扩展引入的噪声超过了其召回率收益, 在原本精度较高的 BM25 上负面影响尤为明显。

PRF 对 BM25 和 Semantic 均有负面作用且延迟显著。BM25 + PRF 的 P@5 为 0.76, 低于基础 BM25 的 0.88; Semantic + PRF 的 P@5 为 0.58, 略高于基础 Semantic 的 0.56。PRF 响应时间方面, TF-IDF + PRF 和 BM25 + PRF 平均耗时约 4.2 – 4.6 秒, 比基础模式慢约两个数量级, 原因在于 PRF 需要执行两轮检索 (初始检索 + Rocchio 反馈后的重检索)。

表 5.4 展示了各配置在每条查询上的 P@5 详细数据。

表 5.4 各查询的 P@5 详细数据 (按配置分列)

查询	TF-IDF	+QE	+PRF	BM25	+QE	+PRF	Sem	+QE	+PRF
climate change	1.00	0.60	1.00	1.00	0.80	0.80	1.00	1.00	1.00
social media	0.60	0.40	0.60	1.00	0.40	1.00	0.60	0.60	0.60
machine learning	1.00	1.00	1.00	1.00	0.80	1.00	1.00	1.00	1.00
covid-19 pandemic	0.00	0.00	0.00	1.00	1.00	0.80	0.80	0.80	0.80
mental health									

续表 5.4

查询	TF-IDF	+QE	+PRF	BM25	+QE	+PRF	Sem	+QE	+PRF
air pollution	0.80	1.00	0.80	1.00	0.80	0.60	0.40	0.40	0.40
urban environment									
supply chain risk	0.40	0.40	0.40	1.00	0.00	0.60	0.60	0.40	0.60
management									
carbon emissions	0.00	0.00	0.00	1.00	0.20	1.00	0.80	0.80	1.00
economic growth									
impact of	0.40	0.00	0.80	0.40	0.00	0.60	0.20	0.00	0.20
monetary policy ...									
artificial intelligence	0.00	0.00	0.00	1.00	0.20	1.00	0.20	0.00	0.20
...									
human mobility	0.20	0.20	0.20	0.40	0.20	0.20	0.00	0.20	0.00
social ...									

从逐查询数据中可以看到：

- BM25 在 10 条查询中有 8 条达到 $P@5 = 1.00$ ，鲁棒性很强。表现较差的是长查询“impact of monetary policy on income inequality” ($P@5 = 0.40$) 和 “human mobility social networks influence” ($P@5 = 0.40$)，前者可能是因为查询含有较多功能词 (of, on)，干扰了 BM25 的词项匹配。
- TF-IDF 在多词查询上表现不稳定，“covid-19 pandemic mental health”、“carbon emissions economic growth” 和 “artificial intelligence sustainable development goals” 的 $P@5$ 均为 0.00，暴露了 TF-IDF 处理复杂查询的局限性。

- 语义检索在“climate change”和“machine learning”上取得了 $P@5 = 1.00$ ，说明在某些主题上语义匹配确实有效；但“human mobility social networks influence”上 $P@5 = 0.00$ ，可能是 Word2Vec 平均池化丢失了关键语义信息。

5.5 查询扩展效果

如表 5.3 所示，WordNet 查询扩展在本次评估中整体表现不理想。BM25 + QE 的 $P@5$ 从 0.88 降至 0.44，降幅达 50%。原因主要有两方面：

- WordNet 同义词扩展会引入多义词的不相关义项，例如“change”的同义词包含“exchange”（兑换）等与“climate change”无关的词项；
- 扩展后的查询词数增多，但新增词项的权重与原始查询词相同，导致检索结果被噪声词项主导。

不过在少数情况下查询扩展仍有正向作用：TF-IDF + QE 在“air pollution urban environment”上 $P@5$ 从 0.80 升至 1.00，说明当同义词指向明确时扩展仍有价值。

5.6 性能分析

表 5.5 展示了各配置的响应时间分布。

表 5.5 各配置的查询响应时间统计

配置	平均时间	范围
TF-IDF	30.7 ms	15.7 – 45.6 ms
TF-IDF + QE	35.9 ms	12.3 – 48.6 ms
TF-IDF + PRF	4573.9 ms	4335 – 4699 ms
BM25	26.2 ms	15.6 – 44.4 ms
BM25 + QE	39.2 ms	19.3 – 62.3 ms
BM25 + PRF	4195.2 ms	4109 – 4290 ms
Semantic	6.5 ms	2.0 – 27.2 ms
Semantic + QE	6.8 ms	2.3 – 29.8 ms
Semantic + PRF	11.6 ms	7.9 – 27.6 ms

三类配置的响应时间有明显的层次差异：

- **Semantic 最快**: 平均 7 – 12 ms, 几乎不受查询扩展影响, PRF 模式也仅增加约 5 ms。原因是语义检索的核心运算是一次矩阵-向量乘法 (17000×100 矩阵), 由 NumPy/BLAS 高效完成, PRF 的额外开销也只是向量空间中的 Rocchio 运算。
- **TF-IDF / BM25 基础和 QE 模式**: 平均 26 – 36 ms, 完全满足实时检索需求。查询扩展带来的额外延迟不大 (约 5 – 13 ms), 主要花在 WordNet 同义词查找上。
- **PRF 模式延迟明显**: TF-IDF + PRF 和 BM25 + PRF 平均耗时约 4.2 – 4.6 秒, 比基础模式慢约 160 倍, 原因在于 PRF 需要两轮完整的倒排索引检索 (初始检索 + Rocchio 反馈后的重检索), 加上中间的文档向量重建。相比之下 Semantic + PRF 仅 11.6 ms, 密集向量空间中做 PRF 的开销几乎可以忽略。

5.7 系统启动性能

表 5.6 展示了系统在线阶段的启动性能与内存占用。

表 5.6 系统启动各阶段耗时与内存占用

阶段	耗时	增量内存	峰值 RSS
索引加载	149.1 ms	+162.2 MB	371.5 MB
元数据加载	10.3 ms	+14.8 MB	386.4 MB
正文库打开	3.1 ms	+0.3 MB	386.7 MB
检索器初始化	416.5 ms	+121.1 MB	507.8 MB

系统从冷启动到就绪的总耗时约 579 ms, 峰值 RSS 为 507.8 MB。各阶段分析如下:

- **索引加载** (149 ms): 将词典 (`lexicon.pkl`) 和 IDF 表 (`idf.pkl`) 反序列化为 Python 对象, 倒排数组 (`postings_doc_ids.npy`、`postings_tfs.npy`) 通过 mmap 映射而非全量读入, 因此增量内存仅 162 MB。
- **元数据加载** (10 ms): 加载 `doc_meta.pkl`, 包含 17,000+ 篇文档的标题、期刊、DOI 等轻量字段, 体积较小。
- **正文库打开** (3 ms): SQLite 数据库仅打开连接, 不预读正文, 增量内存几乎为零。
- **检索器初始化** (417 ms): 主要耗时在于加载语义向量矩阵 (`doc_vectors.npy`, 同样使用 mmap) 和构建检索器对象; TF-IDF 文档模长数组在此阶段加载, 无需重新计算。

整体来看，系统启动时间不到 1 秒，内存峰值约 508 MB，在普通笔记本电脑上可流畅运行。倒排数组和语义向量矩阵均采用内存映射加载，实际物理内存占用由操作系统按需调页决定，冷启动后常驻内存通常低于峰值 RSS。

第6章 项目创新点

在满足课程基本要求的基础上，本项目在算法、系统工程和用户体验方面做了若干额外的探索。

6.1 多模型检索架构

系统同时支持三种检索模型——TF-IDF VSM、BM25 和 Word2Vec 语义检索，提供统一接口和在线切换能力。

除了各模型的独立检索外，Web UI 还提供了**多模型对比**功能：对同一查询同时运行所有模型并排展示结果，并计算结果集间的 Jaccard 相似度，便于直观对比不同检索模型的行为差异。

6.2 语义检索的向量化加速

在语义检索中，本项目没有采用朴素的逐文档循环计算余弦相似度，而是将全部文档向量预组装为一个 $N \times D$ 的 NumPy 矩阵（ N 为文档数， D 为向量维度）。该矩阵以 .npy 文件保存，在线阶段以内存映射方式加载。检索时相似度计算简化为一次矩阵-向量乘法：

$$\vec{s} = \mathbf{M} \cdot \vec{q} \quad (6.1)$$

其中 $\mathbf{M} \in \mathbb{R}^{N \times D}$ 为文档向量矩阵， $\vec{q} \in \mathbb{R}^D$ 为查询向量， $\vec{s} \in \mathbb{R}^N$ 为相似度向量。该操作由底层 BLAS 库（如 OpenBLAS / MKL）执行，相比 Python 层面的 for 循环有显著的速度优势；同时，矩阵文件采用内存映射加载，避免启动时复制整份向量矩阵到普通 Python 对象结构中。

6.3 紧凑索引与内存加载优化

除语义检索外，系统还对在线倒排索引加载进行了结构优化。早期实现将倒排索引保存为 term -> [Posting] 的 Python 对象结构，加载时需要反序列化大量小对象。当

前实现将倒排表拆为词典与连续数组：`lexicon.pkl` 只保存词项到数组切片的映射，`postings_doc_ids.npy` 和 `postings_tfs.npy` 分别保存文档 ID 与词频。在线查询时通过数组切片读取某个词项的倒排列表，BM25、TF-IDF 和 Rocchio 反馈模块均直接遍历数组接口。

这种设计减少了 Python 对象数量，也使倒排数组能够通过 `mmap_mode="r"` 按需映射到进程地址空间。配合构建阶段预计算的 `doc_norms`，TF-IDF 检索器启动时无需再次全量扫描倒排索引重建文档模长。整体上，该优化将在线阶段的加载成本前移到离线构建阶段，更适合包含上万篇文档的本地检索系统。

6.4 基于 WordNet 的查询扩展

查询扩展是提升召回率的常用手段。本项目实现了基于 WordNet^[8] 的自动查询扩展。

WordNet 是一个大型英语词汇语义网络，将词汇组织为同义词集合 (Synset)，建立上下位、整体部分等语义关系。系统利用 WordNet 的同义词关系，为查询中的每个词自动补充语义相近的替代表达，从而在不改变查询意图的前提下扩大检索范围。

该功能设计为可选开关 (CLI 通过 `--query-expansion` 参数，Web UI 通过侧边栏复选框)，用户可根据查询特点灵活选择是否启用。

6.5 分层存储架构

在数据存储方面，本项目将索引、元数据、正文内容和语义向量分层存储，而非放入单一文件：

- **轻量元数据层** (`doc_meta.pkl`)：仅包含标题、期刊、DOI、日期、作者等字段，体积小，系统启动时全量加载到内存；
- **正文内容层** (`doc_texts.db`)：文档的摘要和正文存储在 SQLite 数据库中，通过 `DocTextsDB` 类提供与 Python dict 兼容的 `.get()` 接口，支持按 `doc_id` 按需查询。
- **紧凑倒排层** (`lexicon.pkl`、`postings_doc_ids.npy`、`postings_tfs.npy`)：词典保存数组偏移，倒排项以连续 NumPy 数组存储，在线加载时支持内存映射；

- **语义向量层** (`doc_vectors.npy`、`doc_vector_ids.npy`): 文档语义向量和文档 ID 分离存储, 加载时同样支持内存映射。

该设计使系统启动时仅加载必要的轻量元数据和索引映射, 无需将数百 MB 的正文或大规模向量字典全部载入内存。正文仅在生成 Snippet、词云或关联媒体展示时按需读取, 倒排数组和语义向量矩阵也可以由操作系统按需调页, 降低了内存占用, 使系统在资源有限的环境 (如笔记本电脑) 中能够流畅运行。

6.6 多媒体检索增强

作为多媒体信息检索的初步探索, 系统在搜索结果中集成了词云可视化与关联媒体展示。每条结果旁展示基于该文档摘要自动生成的词云缩略图, 用户不仅能阅读标题和 Snippet, 还可借助词云直观了解文档的主题分布。对于存在关联媒体的论文, 系统还会在结果卡片中展示图片预览、视频链接和补充材料入口, 使检索结果从纯文本列表扩展为包含视觉与媒体线索的呈现形式。

6.7 检索效果评价闭环

本项目不仅在 Web UI 中支持人工评价, 还将评价功能延伸到了命令行:

- **CLI 评价模式**: 通过 `--eval` 参数启用, 检索完成后逐条交互式标注相关性, 实时输出 `Precision@5` 和 `Precision@10`;
- **Web UI 评价**: 通过复选框批量标注, 一键保存;

两端的评价结果统一存储在 `index/evaluations.json` 中, 支持跨会话积累, 按模型分组计算平均 Precision。这使得用户可以在不同场景下 (命令行快速测试、Web 端详细评估) 持续积累评价数据, 为检索效果的量化比较提供依据。

6.8 Rocchio 相关性反馈

本项目实现了经典的 Rocchio 相关性反馈算法^[9], 并针对三种检索模型分别做了适配:

- **TF-IDF VSM**: 在稀疏 TF-IDF 向量空间中直接执行 Rocchio 公式修改查询权重, 通过批量遍历倒排索引重建文档向量以保证效率。
- **Word2Vec 语义检索**: 在密集词向量空间中执行 Rocchio 向量运算, 利用预构建的文档向量矩阵加速质心计算。
- **BM25**: 由于 BM25 没有显式查询向量, 采用基于 Rocchio 权重的查询扩展策略, 选取正反馈文档中高权重新词项追加到查询中。

系统同时支持两种反馈模式: **交互式反馈**由用户逐条标注相关性后迭代优化查询; **伪相关反馈** (PRF) 自动取 Top-N 文档作为正反馈, 无需用户参与。两种模式均集成到 CLI 和 Web UI 中, 用户可调节 Rocchio 参数 (α 、 β 、 γ)。

第 7 章 环境与社会可持续发展

7.1 环境可持续发展考量

系统设计中若有若干举措有助于减少资源消耗：

离线索引与在线检索分离：索引构建是一次性操作，构建完成后持久化到磁盘，后续查询直接加载索引，无需重新处理原始文档，避免了重复计算。

按需查询的存储设计：文档正文存储在 SQLite 数据库中，支持按 doc_id 按需查询，而非将 17,000+ 篇文档正文一次性加载到内存，降低了运行时内存占用。

紧凑索引存储：倒排表和语义向量采用 NumPy 数组与内存映射加载，减少 Python 对象反序列化和常驻内存占用。TF-IDF 文档模长在构建阶段预计算并持久化，避免在线启动时重复扫描全量倒排索引。

倒排索引的检索效率：倒排索引结构使得检索只需访问包含查询词项的倒排列表，而非扫描全部文档。一次查询通常只涉及几百到几千条记录，响应时间在毫秒级。

缓存机制：Web UI 使用 @st.cache_resource 缓存模型实例、@st.cache_data 缓存词云图片，并通过本地 SQLite 缓存关联媒体元数据，避免了重复计算和重复读取。

7.2 社会可持续发展考量

促进知识获取：信息检索系统的核心价值在于帮助用户从海量数据中高效获取所需信息。本系统以 17,000+ 篇跨学科学术论文为语料，为研究者提供了便捷的文献检索工具。

开放技术栈：系统完全基于开源工具构建（Python、NLTK、Gensim、Streamlit 等），不依赖商业软件或付费 API。数据处理全部在本地完成，不涉及用户数据的外部传输。

可解释性：TF-IDF 和 BM25 算法具有明确的数学定义，用户可通过 Snippet 高亮和相关度分数理解排序依据，避免了黑箱算法可能带来的不透明问题。

第 8 章 总结与展望

8.1 主要工作

本项目设计并实现了 IREngine 学术论文信息检索系统，完成了以下工作：

1. 实现了完整的文档解析与预处理流程，从 17,576 篇 .txt 格式学术论文中提取结构化信息并做标准化处理；
2. 自主实现了紧凑倒排索引，支持词频、文档频率和 IDF 统计，并通过连续数组与内存映射优化在线加载；
3. 实现了 TF-IDF 向量空间模型和 BM25 概率检索模型；
4. 探索了基于 Word2Vec 的语义检索和基于 WordNet 的查询扩展；
5. 实现了基于 Rocchio 算法的相关性反馈，覆盖三种检索模型的伪相关反馈和交互式反馈；
6. 提供了 CLI 和 Streamlit Web UI 两种界面，均支持人工相关性标注 (Precision@K)；
7. 集成了词云可视化和 DOI 关联媒体展示，支持在检索结果中呈现图片、视频链接和补充材料入口；
8. 编写了覆盖核心模块的单元测试 (51 个用例全部通过)。

8.2 不足与展望

受限于个人时间和资源，系统仍有以下不足：

1. **评价指标单一**：当前仅支持 Precision@K，后续可扩展 MAP、NDCG 等指标；
2. **语义表示较简单**：Word2Vec 平均池化的效果有限，可考虑引入 Sentence-BERT 等预训练模型获取更好的语义向量；
3. **索引不支持增量更新**：目前为全量重建模式，后续可实现增量索引以支持新文档的动态加入；

参考文献

- [1] MANNING C D, RAGHAVAN P, SCHÜTZE H. Introduction to Information Retrieval[M]. Cambridge University Press, 2008.
- [2] PORTER M F. An Algorithm for Suffix Stripping[J]. Program, 1980, 14(3): 130-137.
- [3] SPÄRCK JONES K. A Statistical Interpretation of Term Specificity and Its Application in Retrieval[J]. Journal of Documentation, 1972, 28(1): 11-21.
- [4] SALTON G, WONG A, YANG C S. A Vector Space Model for Automatic Indexing[J]. Communications of the ACM, 1975, 18(11): 613-620.
- [5] ROBERTSON S E, ZARAGOZA H. The Probabilistic Relevance Framework: BM25 and Beyond[J]. Foundations and Trends in Information Retrieval, 2009, 3(4): 333-389.
- [6] ROBERTSON S E, WALKER S. Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval[J]. Proceedings of the 17th Annual International ACM SIGIR Conference, 1994: 232-241.
- [7] MIKOLOV T, CHEN K, CORRADO G, et al. Efficient Estimation of Word Representations in Vector Space[J]. arXiv preprint arXiv:1301.3781, 2013.
- [8] MILLER G A. WordNet: A Lexical Database for English[J]. Communications of the ACM, 1995, 38(11): 39-41.
- [9] ROCCHIO J J. Relevance Feedback in Information Retrieval[M]//The SMART Retrieval System: Experiments in Automatic Document Processing. Prentice-Hall, 1971: 313-323.
- [10] BAEZA-YATES R, RIBEIRO-NETO B. Modern Information Retrieval: The Concepts and Technology behind Search[M]. 2nd ed. Addison-Wesley, 2011.
- [11] PENNINGTON J, SOCHER R, MANNING C D. GloVe: Global Vectors for Word Representation[C]//Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP). 2014: 1532-1543.

附录 A 系统使用说明

A.1 环境安装

本系统使用 Python 3.11+ 开发，推荐使用 uv 包管理器进行依赖安装：

```
1 uv sync
```

 Bash

主要依赖包括：NLTK(文本处理)、NumPy(数值计算)、Gensim(Word2Vec)、Streamlit (Web UI)、wordcloud + matplotlib (词云生成)。

A.2 索引构建

```
1 # 基础索引
```

 Bash

```
2 uv run python build_index.py --data-dir data --index-dir index
```

```
3
```

```
4 # 同时训练语义模型
```

```
uv run python build_index.py --data-dir data --index-dir index --  
5 semantic
```

A.3 命令行检索

```
1 # BM25 检索 (默认)
```

 Bash

```
2 uv run python cli.py
```

```
3
```

```
4 # 语义检索 + 查询扩展 + 人工评价
```

```
uv run python cli.py --model semantic --query-expansion --eval --  
5 top-k 10
```

A.4 Web 界面

```
1 uv run streamlit run app.py
```

 Bash

A.5 运行测试

```
1 uv run python -m pytest tests/ -v
```



附录 B 项目文件结构

1	IREngine/	
2	— app.py	# Streamlit Web UI
3	— cli.py	# 命令行检索界面
4	— build_index.py	# 索引构建脚本
5	— build_media_index.py	# 关联媒体缓存构建脚本
6	— benchmark.py	# 自动化评测脚本 (9 种配置 × 10 查询)
7	— pyproject.toml	# 项目配置与依赖
8	— src/	
9	— parser.py	# 文档解析器
10	— preprocessor.py	# 文本预处理
11	— indexer.py	# 倒排索引
12	— retriever.py	# TF-IDF / BM25 检索与查询扩展
13	— semantic.py	# Word2Vec 语义检索
14	— feedback.py	# Rocchio 相关性反馈
15	— snippet.py	# Snippet 生成与高亮
16	— evaluator.py	# 人工评价系统
17	— storage.py	# SQLite 文档存储
18	— media.py	# DOI 关联媒体缓存与过滤
19	— wordcloud_gen.py	# 词云生成
20	— ui_styles.py	# Web UI 样式定义
21	— tests/	
22	— conftest.py	# 测试 fixtures (共享索引实例)
23	— test_preprocessor.py	
24	— test_indexer.py	
25	— test_retriever.py	
26	— test_semantic.py	
27	— test_feedback.py	
28	— test_media.py	
29	— test_benchmark.py	
30	— test_app_ui.py	
31	— data/	# 原始论文数据 (.txt)
32	— index/	# 索引与模型文件
33	— lexicon.pkl	# 词典与倒排偏移

34	— idf.pkl	# IDF 值
35	— postings_doc_ids.npy	# 倒排文档 ID 数组 (mmap)
36	— postings_tfs.npy	# 倒排词频数组 (mmap)
37	— doc_meta.pkl	# 文档元数据
38	— doc_texts.db	# SQLite 正文库
39	— doc_vectors.npy	# 语义向量矩阵 (mmap)
40	— doc_vector_ids.npy	# 向量对应的文档 ID
41	— media_assets.db	# DOI 关联媒体缓存 (SQLite)